

YOLO V5

Structured Notes

Architecture, CSP Backbone, FPN

Neck, Detection Head,
SPPF, YOLO Label Format, and Practical Training Pipeline

Contents

1	Introduction to YOLO V5	2
1.1	Applications of YOLO V5	2
2	YOLO V5 Architecture Overview	2
3	Backbone: Feature Extraction with CSPNet	3
3.1	Feature Hierarchy	3
3.2	Backbone Layer Structure	3
4	C3 Block: Cross Stage Partial Bottleneck	3
4.1	C3 Block with Skip Connection (Backbone)	4
4.2	C3 Block without Skip Connection (Neck)	4
4.3	Why Different Convolution Operations Produce Different Outputs	4
5	SPPF: Spatial Pyramid Pooling Fast	4
5.1	SPPF Operation	5
6	Neck: Feature Pyramid Network (FPN)	5
6.1	Purpose of the Neck	5
6.2	Why Concatenate Lower and Higher Level Features	5
6.3	Neck Layer Structure	5
6.4	Cross-Layer Connections	6
7	Head: Object Detection	6
7.1	Anchor Boxes	6
8	YOLO Label Format	6
8.1	Comparison: YOLO Format vs XML Format	7
8.2	Creating YOLO Format Labels	7
9	YOLO V5 Training Pipeline	7
9.1	Environment Setup	7
9.2	Dataset Preparation	7
9.3	Custom Data YAML Configuration	7
9.4	Model Creation and Training	8
9.5	Weights and Inference	8
10	Summary	8

1 Introduction to YOLO V5

YOLO V5

YOLO V5 (You Only Look Once, Version 5) is a real-time object detection model that detects objects in images by drawing bounding boxes around them. It represents the fifth iteration of the YOLO family and was introduced to address limitations of earlier versions.

YOLO V5 improves upon earlier versions (V1, V2, V3, V4) in several key areas:

- **Higher speed** — faster inference compared to previous versions
- **Better accuracy** — improved detection performance overall
- **Small object detection** — earlier versions (particularly V1) struggled with small objects and produced incorrect detections
- **Adverse conditions** — handles low contrast, poor lighting, crowded scenes, and occluded objects better than predecessors

Limitations of YOLO V1

YOLO V1 worked well when the object size was large enough, but it failed in cases where:

- Objects were small in size
- Environmental contrast was low
- Lighting conditions were adverse
- Objects appeared in crowded scenes or alongside other objects

These limitations motivated the development of YOLO V5.

1.1 Applications of YOLO V5

YOLO V5 has widespread applications across various domains:

- **Surveillance systems** — real-time monitoring and threat detection
- **Autonomous driving** — detecting vehicles, pedestrians, and obstacles
- **Face detection** — identifying and localizing faces in images
- **Medical imaging** — detecting anomalies in medical scans

2 YOLO V5 Architecture Overview

YOLO V5 comprises three major architectural components:

YOLO V5 Architecture = Backbone + Neck + Head

1. **Backbone** — Extracts features at multiple levels (low, mid, high)
2. **Neck** — Concatenates features across different scales for multi-scale detection
3. **Head** — Performs final object detection with bounding boxes

Component	Key Module	Function
Backbone	CSPNet (C3 blocks)	Feature extraction
Backbone	SPPF	Multi-scale pooling
Neck	FPN + Upsampling	Feature concatenation across scales
Head	Detection layer	Bounding box prediction

3 Backbone: Feature Extraction with CSPNet

Backbone

The **backbone** is responsible for extracting features from the input image. It uses multiple stages of **Cross Stage Partial Networks (CSPNet)** to progressively extract features from low-level to high-level representations.

3.1 Feature Hierarchy

The backbone extracts three levels of features:

1. **Low-level features** — edges, textures, contours in the image
2. **Mid-level features** — parts of objects (e.g., face, arms, legs of a human)
3. **High-level features** — complete object understanding and recognition

As the network goes deeper, it progressively extracts higher-level features. The backbone also reduces computational cost because only a few layers are responsible for building each stage of the network.

3.2 Backbone Layer Structure

The backbone is a sequential model where the output of each preceding layer serves as input to the succeeding layer. The layer configuration in the YOLO V5s YAML blueprint follows this pattern:

Layer	Repeat	Module	Parameters
1	1	Conv	64 filters, 6×6 , stride 2, padding 2
2	1	Conv	128 filters, 3×3 , stride 2, padding 1
3	3	C3	128 filters
4	1	Conv	256 filters, 3×3 , stride 2, padding 1
5	6	C3	256 filters
6	1	Conv	512 filters, 3×3 , stride 2
7	9	C3	512 filters
8	1	Conv	1024 filters, 3×3 , stride 2, padding 1
9	3	C3	1024 filters
10	1	SPPF	1024 filters, kernel size 5

YAML Blueprint Notation

Each layer in the YAML file is specified as: `[from, repeat, module, args]`

- **from** = -1 means the layer takes input from the **previous layer**
- **repeat** specifies how many times the layer/block is repeated
- **module** is the operation type (Conv, C3, SPPF, etc.)
- **args** contains filter count, kernel size, stride, and padding

4 C3 Block: Cross Stage Partial Bottleneck

C3 Block

The **C3 block** (Cross Stage Partial Bottleneck) is the core building block used in both the backbone and neck of YOLO V5. It performs feature extraction through a combination of convolution operations, residual (skip) connections, and concatenation. The C3 block is defined in `common.py`.

4.1 C3 Block with Skip Connection (Backbone)

The C3 block used in the backbone includes a skip connection (residual connection). The operation proceeds as follows:

C3 Block Operation (with skip connection):

1. Take input X
2. Perform convolution on X to get Y_1
3. Perform convolution on Y_1 to get Y_2
4. Add Y_2 back to input: $Y' = Y_2 + X$ (skip/residual connection)
5. Perform a separate convolution on X to get Y_3 (parallel path)
6. Concatenate Y' and Y_3 : $Y_4 = \text{concat}(Y', Y_3)$
7. Perform convolution on Y_4 to get the final output Y_5

The skip connection (adding Y_2 back to X) is the **bottleneck** operation. It helps preserve information from the original input and addresses the vanishing gradient problem.

4.2 C3 Block without Skip Connection (Neck)

In the neck portion of the network, the C3 block is used with **False** flag, meaning **no skip connection** is used. The operation becomes:

C3 Block Operation (without skip connection):

1. Take input X
2. Perform convolution on X to get Y_1
3. Perform convolution on Y_1 to get Y_2
4. Perform convolution on Y_2 to get Y_3 (no addition back to X)
5. Perform a separate convolution on X to get X_1 (parallel path)
6. Concatenate Y_3 and X_1
7. Perform convolution on the concatenation to get the final output

Key Difference

The C3 block with skip connection (**True**) is used in the **backbone**. The C3 block without skip connection (**False**) is used in the **neck**. The difference is whether Y_2 is added back to the original input X or not.

4.3 Why Different Convolution Operations Produce Different Outputs

Even when two parallel convolution operations use the same number of filters (e.g., 64 filters), they produce different outputs because:

- Filters are **initialized with different random numbers**
- Different filters extract different features (e.g., one may extract edges while another extracts contours)
- Each convolution operation is independent with its own learned weights

5 SPPF: Spatial Pyramid Pooling Fast

SPPF

SPPF (Spatial Pyramid Pooling Fast) is a block at the end of the backbone that performs max pooling at multiple scales and concatenates the results. It enables the network to capture features

at different spatial resolutions.

5.1 SPPF Operation

SPPF Block Operation:

1. Take input X , perform convolution to get X'
2. Perform max pooling on X' to get Y_1
3. Perform max pooling on Y_1 to get Y_2
4. Perform max pooling on Y_2 to get Y_3
5. Concatenate along the channel dimension: $Y_4 = \text{concat}(X', Y_1, Y_2, Y_3)$
6. Perform convolution on Y_4 to get the final output Y_5

The SPPF block processes features at **different scales** through successive max pooling operations. Each max pooling reduces the spatial resolution while preserving the most prominent features, and by concatenating features from all scales, the network gains a multi-scale understanding of the image.

With SPPF, the backbone portion of the network is complete.

6 Neck: Feature Pyramid Network (FPN)

Neck

The **neck** portion of YOLO V5 uses **Feature Pyramid Networks (FPN)** to concatenate features from different stages of the backbone. This enables the model to detect objects of varying sizes — small, medium, and large.

6.1 Purpose of the Neck

The neck concatenates features across different levels to achieve:

- **Accurate localization** — determining where the object is in the image
- **Correct object understanding** — identifying what type of object is present
- **Multi-scale detection** — handling small, medium, and large objects

6.2 Why Concatenate Lower and Higher Level Features

Higher-level layers process features extracted by lower layers and perform additional processing for complete object recognition. However, during this processing, some lower-level features may get lost. Concatenation recovers these lost features and provides a more complete representation.

6.3 Neck Layer Structure

The neck uses several key operations:

1. **Convolution layers** with 1×1 filters to reduce channel dimensions (e.g., 512 filters to 256)
2. **Upsampling** by a factor of 2 using **nearest neighbor** interpolation — for each pixel, the nearest neighbors are filled with the same value to increase spatial resolution
3. **Concatenation** of outputs from the current layer with outputs from earlier backbone layers along the channel dimension
4. **C3 blocks without skip connections** (False flag) for further feature processing

Upsampling in the Neck

The upsampling operation uses **nearest neighbor** algorithm with a scale factor of 2. The output size is not fixed (**None** in the configuration) because it depends on the input dimensions at each

upsampling layer. This is distinct from convolution — upsampling increases spatial resolution rather than extracting features.

6.4 Cross-Layer Connections

The neck takes outputs from specific backbone layers and concatenates them with the current processing path. For example:

- One concatenation layer combines the upsampled output with the 6th layer (backbone) output
- Another concatenation layer combines the upsampled output with the 4th layer (backbone) output
- Further concatenation layers link with layers 10 and 14 in the neck itself

All concatenations happen along the **channel dimension**.

7 Head: Object Detection

Head

The **head** is the final component of YOLO V5 that performs actual object detection. It takes outputs from multiple neck layers and produces bounding box predictions along with class probabilities.

The head takes the output from layers 17, 20, and 23 of the network. It performs detection using:

- **Bounding boxes** — rectangular regions drawn around detected objects
- **Number of classes (nc)** — the total number of object categories (e.g., 1 for a single-class fish detection task)
- **Anchors** — predefined bounding box dimensions for different object sizes

7.1 Anchor Boxes

Anchors are predefined bounding box sizes that serve as initial references for detection. They are defined for three object scales:

Object Scale	Anchor Sizes (width × height)		
Small objects	10 × 13,	16 × 30,	33 × 23
Medium objects	30 × 61,	62 × 45,	59 × 119
Large objects	116 × 90,	156 × 198,	373 × 326

Each grid cell in the image uses three bounding boxes (anchors) for each scale. This allows the model to detect objects of varying sizes within the same image.

8 YOLO Label Format

YOLO Format Labels

The **YOLO format** for labels stores bounding box annotations in a simple text file. Each line represents one object with the format:

```
class_id  x_center  y_center  width  height
```

All coordinates are normalized relative to the image dimensions.

8.1 Comparison: YOLO Format vs XML Format

Aspect	YOLO Format	XML Format
File type	.txt	.xml
Coordinate format	x_center, y_center, width, height	x_min, y_min, x_max, y_max
Normalization	Normalized (0–1)	Pixel values
Class representation	Integer ID (0, 1, 2, ...)	Class name string
Simplicity	Simple, compact	Verbose, structured
Used with	YOLO V5 and later	YOLO V1, Pascal VOC

Class ID

The class ID starts at 0. For a single-class detection task (e.g., fish detection), the class ID is always 0. For multi-class tasks, classes are numbered sequentially: 0, 1, 2, etc.

8.2 Creating YOLO Format Labels

Labels can be created using annotation tools like **makesense.ai**:

1. Load images into the annotation tool
2. Draw bounding boxes around target objects
3. Export annotations in **YOLO format**
4. The raw coordinates (**x_min, y_min, x_max, y_max**) are converted to (**x_center, y_center, width, height**) through code

9 YOLO V5 Training Pipeline

9.1 Environment Setup

Clone YOLO V5 Repository

```

1 # Clone the YOLO V5 repository
2 !git clone https://github.com/ultralytics/yolov5
3
4 # This installs all libraries and dependencies
5 # required for YOLO V5

```

Listing 1: Installing YOLO V5 dependencies

9.2 Dataset Preparation

The training pipeline requires the following steps:

1. **Upload training data** — upload the zipped training data to Google Colab
2. **Unzip the data** — extract to a folder and provide the correct path
3. **Upload custom data YAML** — place the `custom_data.yml` configuration file inside the `yolov5/data/` directory
4. **Verify folder structure** — ensure the training data folder (`train_data`) contains images and corresponding YOLO format label files

9.3 Custom Data YAML Configuration

The `custom_data.yml` file defines:

- **Anchor sizes** — predefined bounding box dimensions for small, medium, and large objects

- **Number of classes (nc)** — total number of object categories
- **Class names** — list of class labels
- **Data paths** — paths to training and validation image directories

9.4 Model Creation and Training

Model Blueprint

The YOLO V5 model is created from a **blueprint** defined in a YAML file (e.g., `yolov5s.yaml`). This blueprint specifies the complete architecture — all layers, their configurations, and connections. The `DetectionModel` class reads this blueprint and constructs the neural network.

The training process follows these steps:

1. The `train.py` script is executed with arguments specifying the custom data YAML, model weights, and other hyperparameters
2. The `DetectionModel` class builds the model from the YAML blueprint
3. **Data loaders** load the training images and labels in batches
4. For each epoch, the model processes the data, computes a **loss**, and updates the weights using **optimizers**
5. **Checkpoints** are saved — both the last epoch weights and the best epoch weights

9.5 Weights and Inference

After training, two sets of weights are saved:

- **Last weights** — weights from the final training epoch
- **Best weights** — weights from the epoch with the best performance

For inference (testing), the **best weights** should be used. The path to the best weights is copied and provided to the detection script along with the test image path.

Inference with Best Weights

```
1 # Copy the path of the best weights
2 # best_weights_path = "runs/train/exp/weights/best.pt"
3
4 # Run detection on a test image
5 !python detect.py --weights {best_weights_path} \
6                   --source {test_image_path}
```

Listing 2: Running detection on test images

10 Summary

- **YOLO V5** is a real-time object detection model that improves upon earlier versions with better speed, accuracy, and small object detection capabilities
- The architecture consists of three components: **Backbone** (feature extraction), **Neck** (multi-scale feature fusion), and **Head** (detection)
- The **Backbone** uses Cross Stage Partial Networks (CSPNet/C3 blocks) to extract low-level, mid-level, and high-level features, ending with an SPPF block for multi-scale pooling
- The **C3 block** combines convolution, skip connections (in backbone) or no skip connections (in neck), and concatenation operations
- **SPPF** performs successive max pooling operations at different scales and concatenates the results for multi-resolution feature representation
- The **Neck** uses Feature Pyramid Networks with upsampling and cross-layer concatenation to enable detection of small, medium, and large objects

- The **Head** uses anchor boxes of predefined sizes to detect objects and draw bounding boxes
- **YOLO format labels** use `class_id`, `x_center`, `y_center`, `width`, `height` (normalized), as opposed to the XML format used in earlier versions
- The training pipeline involves cloning the YOLO V5 repository, preparing labeled datasets, configuring a custom data YAML, and training with checkpointed best weights